

RepViT 轻量图像分类模型复现、优化与多 模型部署

选 ONNX Runtime 1.30.0：1.29.0 / 1.29.1 存在 CPU FP16 Gemm 回退缺陷，会污染基准测试；

完整快照 [outputs/env_snapshot.json](#)。

目 录

一、任务背景与复现范围	4
二、RepViT 论文核心思路	5
三、RepViT-M0.9 结构	6
四、官方预训练模型评价	6
五、多型号规模和性能比较	9
六、数据集与数据划分	10
七、Baseline 迁移训练	11
八、优化方法与实验假设	12
九、定量结果	13
十、曲线与混淆矩阵分析	14
十一、Grad-CAM 与失败案例	17
十二、结构重参数化	20
十三、ONNX 多模型部署	22
十四、性能测试	24
十五、遇到的问题和解决方法	24
十六、总结与后续计划	26
附录 A：实测发现并修复的代码缺陷	27
附录 B：产物索引	28
附录 C：鲁棒性测试（7 类扰动 × 6 级 severity）	28
附录 D：深入可解释性	29
附录 E：集显部署（进阶④）—— 本机不具备条件	29
附录 F：跨数据集分布与外部实拍图（§11.5 的完整数据）	29
附录 G：多分支融合的零填充细节与单 Block 示例（§12.2 / §12.6 的展开）	30
附录 H：消融交互、噪声判据与延迟/MACs 分析（§9.1 / §14.4 的完整数据）	30

2026 秋 one 团队 AI 算法组考核报告

一、任务背景与复现范围

1.1 任务背景

阅读并理解论文《RepViT: Revisiting Mobile CNN From ViT Perspective》（arXiv:2307.09283, CVPR 2024），基于官方开源项目完成 RepViT 的运行评价、迁移训练、模型优化、结果可视化、结构重参数化与 ONNX 部署。论文核心论点：RepViT 名称虽含 "ViT"，但**不含自注意力模块**，其性能来自轻量 ViT 的宏观架构选择与训练策略。

1.2 复现范围

范围	内容
做	官方 ImageNet-1K 权重在 ImageNetV2 matched-frequency 确定性固定子集（1000 张，1000 类各 1 张）上的评价（5 个型号）；M0.9 在 Oxford-IIIT Pet 37 类的迁移训练；4 组控制变量优化 + 3 组组合消融；曲线/混淆矩阵/Grad-CAM；结构重参数化数值验证；5 个 ONNX 模型（4 个官方 + 1 个 Pet-37）的 CPU 部署与性能测试
不做	不在完整 ImageNet-1K 上从头训练；不追求论文 300/450 epoch 精度；不做第八章拓展任务（INT8 量化、知识蒸馏、剪枝、多随机种子、边缘设备部署、核心模块独立实现）

1.3 环境指纹

项	值
Python / torch / torchvision	3.14.5 / 2.14.0+cu126 / 0.29.0+cu126
timm / ONNX / ONNX Runtime	1.0.29 / 1.22.0 / 1.30.0 (CPUExecutionProvider)
GPU / CPU / 内存 / OS	RTX 4060 Laptop 8 GB (CUDA 12.6) / i7-13650HX (20 逻辑核) / 15.8 GB / Windows 11 10.0.26200

1.4 三个必须区分的数字口径

同一模型在不同口径下的参数量**都是对的**，不标口径就是错的：

口径	M0.9 参数量	说明
训练态双头（含蒸馏头，C=1000）	5,489,328	timm 裸 create_model 默认
未融合单头（C=1000 / C=37）	5,103,560 / 4,732,805	官方 README 的 5.1M / 本任务训练态
骨干 / 融合后单头（C=1000 / C=37）	4,717,792 / 5,067,056 / 4,696,301	骨干不含分类头；融合后 = 结构重参数化之后

MACs 亦有 thop 与 fvcare 两种口径：thop = **847,050,816** (0.847 GMACs) ,

fvcare = **832,165,824** (同口径, **不是 2 倍**)。落盘 `outputs/metrics/count_params.json`、`count_flops.json`。

二、RepViT 论文核心思路

2.1 核心论点

轻量 ViT 在移动端的优势通常被归功于多头自注意力 (MHSA)。论文用**增量式架构改造实验**证明:

真正起作用的是轻量 ViT 的**宏观架构与训练策略**, 而非 MHSA——从 MobileNetV3 出发, 每一步只引入一处轻量 ViT 的高效设计, 最终得到 RepViT 家族。因此 RepViT 名字里虽有 "ViT", 却**没有任何自**

注意力算子: 全局信息交互由 1×1 卷积 (Channel Mixer) 与 depthwise 卷积 (Token Mixer) 完成, 感受野靠堆叠与下采样扩大。ONNX 图中可直接验证: 算术节点只有 Conv / Gemm / Add / Mul / Div / Relu 等, **没有 MatMul、没有用于注意力的 Softmax**。

2.2 关键设计逐条

- **Token / Channel Mixer 分离**: 每个 Block 先空间混合再通道混合、中间加残差——与 ViT 的 MHSA + FFN 对应, 容量可独立调节 (不像倒残差把两者耦合)。
- **RepVGWDW Token Mixer**: 训练态 = $3\times 3 + 1\times 1$ depthwise 双分支 + BN, 推理态融合成单个 3×3 depthwise——训练时表达力更强、推理时零开销 (见第十二节)。
- **降低扩张比 + 加宽**: 扩张比由 MobileNetV3 的 6 降到约 2, 同时整体加宽——减少逐点卷积 MACs, 把预算换成宽度, 同等 MACs 下精度更高。
- **Early Stem / 更深的下采样 / 选择性 SE**: 两组 stride=2 的 3×3 卷积代替 ViT 的 patchify (大核非重叠卷积、局部纹理建模弱); 下采样放更深层级 (避免过早丢失细粒度信息); 仅部分 Block 后加 SE (全局池化 + 两次全连接有延迟代价)。
- **简单分类头**: 单个 BN + Linear (**NormLinear**)——复杂头是实打实的推理延迟, 小模型上精度收益有限。
- **depthwise + pointwise 分工**: depthwise 管空间 ($\approx k^2C$)、pointwise 管通道 ($\approx C^2$)——把标准卷积的 C^2k^2 降到 $C^2 + k^2C$, 移动端标准拆解。

2.3 五个型号的差别

同一套 Block 在不同宽度/深度上缩放, 差别集中在各 stage 的通道数与 Block 数;

实测 M0.9 的 4 个 stage Block 数为 **[2, 2, 14, 2]** (共 20 个)。

三、RepViT-M0.9 结构

下图由 `tools/draw_arch.py` 用 forward hook 现测真实张量形状生成（非论文插图、非手画）。

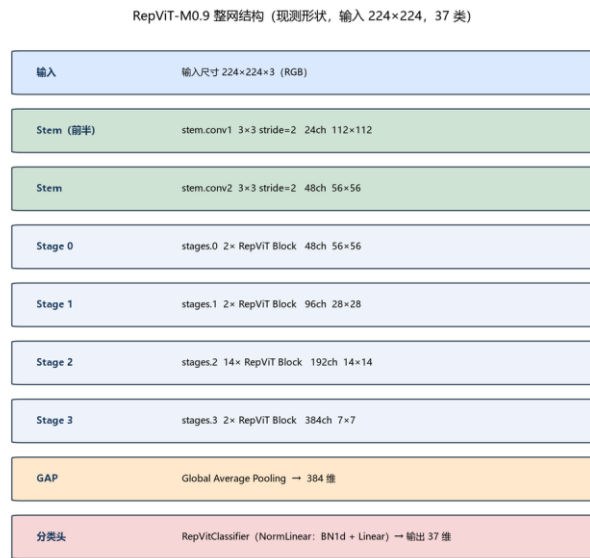


图 3-1 RepViT-M0.9 整网结构与 9 要素（输入 224、Stem、四个阶段、分辨率 224→112→56→28→14→7、通道

3→24→48→96→192→384、RepViT Block、GAP、分类头、输出 37 维）。数据源

`outputs/architecture/repvit_m0_9_arch.png`, 由 `tools/draw_arch.py` 用 forward hook 现测张量形状生成（文字版同目录 `repvit_m0_9_arch.md`）。

```

输入 224×224×3 (RGB)
Stem stem.conv1 3×3 s2 → 24ch 112×112 ; stem.conv2 3×3 s2 → 48ch 56×56 ← Early Convolution Stem
Stage 0 stages.0 2× Block 48ch 56×56 | Stage 1 stages.1 2× Block 96ch 28×28
Stage 2 stages.2 14× Block 192ch 14×14 | Stage 3 stages.3 2× Block 384ch 7×7
GAP → 384 维 ; Head RepViTClassifier → NormLinear(BatchNorm1d + Linear) → 37 维
  
```

- 分辨率 224 → 112 → 56 → 28 → 14 → 7（总下采样 32×）；通道 3 → 24 → 48 → 96 → 192 → 384。
- **Stage 2 独占 14 个 Block**（共 20 个），与论文「第三阶段布置更多 Block」一致。
- 每个 Block = Token Mixer（RepVGGDW：3×3 + 1×1 depthwise 双分支）+ Channel Mixer（1×1 升维 → GELU → 1×1 降维）+ 残差，部分 Block 带 SE。
- **训练态 vs 推理态**：训练态 Block 内含多分支与 BN；推理态融合后 Token Mixer 变成单个 3×3 depthwise 卷积。产物：`outputs/architecture/repvit_m0_9_arch.png`。

四、官方预训练模型评价

4.1 评价口径

评价集 = ImageNetV2 matched-frequency 确定性固定子集 1000 张（1000 类各 1 张），清单

`datasets/lists/imagenetv2_mf_1000.txt`（1000 行 / 87,780 B, sha256 `d5532205...6c9d05`）。

选取规则：类别下标 0..999 升序，每类目录内 `sorted()` 取文件名第一个，**无随机种子** → 任何人可

复现出逐字节相同的清单。出处：HF [vaishaal/ImageNetV2](#) 的 [imagenetv2-matched-frequency.tar.gz](#) (1,264,079,360 B, sha256 [f0c37fdf...c9ca7c](#)，与 HF [X-Linked-ETag](#) 逐字符相同；MIT；上游 [modestyachts/ImageNetV2](#))。预处理：[Resize\(256, bicubic\) → CenterCrop\(224\) → Normalize\(ImageNet\)](#)，即 [crop_pct = 0.875](#)（官方口径）。权重 / 实现：官方 Releases v1.0 的 [*_distill_300e.pth](#) + vendored 官方实现 + [replace_batchnorm](#) 融合态。设备：RTX 4060 Laptop, FP32, batch=64。

口径声明：① ImageNetV2 是 Recht et al. (NeurIPS 2019) 发布的新采集测试集，不是

[ImageNet-1K 验证集](#)、也不是考核方下发的清单，下列结果只代表「该固定子集上的实际运行结果」；

② 它与论文/官方在 ImageNet-1K 上公布的值**不可直接比较**（不得并列或相减）——基准本身更难，

低 10~15 个百分点是**基准性质**，不是模型退化。出处与哈希落盘 [report/IMAGENETV2_PROVENANCE.md](#)。

4.2 实测结果

5 个型号在 ImageNetV2 固定子集（1000 张）上的 Top-1 / Top-5：**M0.9 = 68.70 / 85.70**、

M1.0 = 69.20 / 86.40、M1.1 = 70.80 / 87.20、M1.5 = 71.40 / 89.00、M2.3 = 73.60 / 89.90

（含参数量、ONNX 文件大小与延迟的完整表见 §5，逐型号落盘

[outputs/pretrained_eval/<model>/metrics.json](#)）。

预处理口径对照（同批图片，只改 crop_pct，落盘 [metrics.json](#) 的 [crop_pct_sweep](#)）：

M0.9 = 68.70/85.70 (0.875) vs 68.30/87.00 (0.95)；M1.0 = 69.20/86.40 vs 69.40/87.50；

M1.1 = 70.80/87.20 vs 71.00/87.70；M1.5 = 71.40/89.00 vs 73.30/89.60；M2.3 = 73.60/89.90 vs

74.20/89.70。两套口径**不可混用**；主口径先验固定为 0.875，未据任何结果做超参选择。

4.3 七类结果来源（逐类分开，不得互相替代）

#	来源类别	数据与口径	关键结果	依据产物
1	论文公布	ImageNet-1K, 1000 类, M0.9 300e 蒸馏	Top-1 78.7 % , 5.1 M（融合后单头），0.8 G, iPhone 12 + CoreML 0.9 ms	report/sources/literature.yaml#paper
2	官方仓库公布	ImageNet-1K, 1000 类, README Model Zoo	Top-1 78.7 % / Top-5 79.1 % , 5.1 M, 0.8 G	report/sources/literature.yaml#official_repo
3	官方权重实际运行	ImageNetV2 固定子集 1000 张, 本机 PyTorch CUDA	M0.9 68.70/85.70 、M1.0 69.20/86.40、M1.1 70.80/87.20、M1.5 71.40/89.00、M2.3 73.60/89.90	outputs/pretrained_eval/*/metrics.json
4	自行训练 Baseline	Pet 37 类 test（3669 张, 唯一一次评价）	Top-1 92.34 % 、Top-5 99.26 % 、Macro-F1 92.22 %	outputs/metrics/baseline_test.json

#	来源类别	数据与口径	关键结果	依据产物
5	自行优化模型	同划分与评价方式，组合方案 <code>opt_combo</code>	Top-1 92.12 % 、Top-5 99.59 % 、 Macro-F1 92.01 %	<code>outputs/metrics/opt_combo_test.json</code>
6	PyTorch 推理	batch=1 / FP32 / CUDA / <code>threads=4</code> / 预热 10 + 正式 50	M0.9 融合后 mean 8.41 / P50 8.26 / P95 9.93 ms (未融合 17.15, 融合收益 -51.0 %) ; M1.0 mean 8.25 / P50 8.09 / 未融合 15.46 (-46.6 %)	<code>outputs/pretrained_eval/*/latency.json</code>
7	ONNX 部署	ORT 1.30.0 CPUExecutionProvider / batch=1 / FP32 / 4 线程	P50 7.68 (M0.9) ~ 34.41 (M2.3) ms; 6 模型全表见 §14.2	<code>outputs/benchmarks/summary.csv</code>

第 1、2 类是 ImageNet-1K 公布值，第 3 类是 ImageNetV2 实跑值，两者不同数据集、不可比；

第 4、5 类是 Pet-37 (37 类口径)；第 6、7 类是同一批权重在两个后端上的延迟口径。

第 6 行的「融合收益」即结构重参数化在 PyTorch 侧的收益（官方 iPhone 12 的 0.9 ms 与本机不可直接比较：设备、框架、精度、协议全不同）。

4.5 Top-5 样例与正误案例（2 正 + 2 误）

每型号落盘 ≥6 张图片的 Top-5 类别与置信度，并配案例图（`outputs/pretrained_eval/<model>/`
下的 `top5_samples.json` 与 `cases/`）。下表 4 条数字全部取自 `repvit_m0_9` 的落盘产物
（同源汇总：`outputs/report_assets/table_cases.{csv,md}`）。

案例 (<code>repvit_m0_9/cases/</code>)	真值	预测 Top-1	置信度	Top-5 其余四项	归因
正确 1 <code>correct_1.png</code>	Band Aid	Band Aid	1.0000	carton / toilet tissue / packet / oil filter	主体居中、结构清晰，余项同属「带状耗材」语义群
正确 2 <code>correct_2.png</code>	punching bag	punching bag	0.9999	sleeping bag / rain barrel / balloon / barbell	悬挂姿态与背景干净，判别性部位完整可见
错误 1 <code>wrong_1.png</code>	sarong	umbrella	0.9990	freight car / mountain tent / breakwater / parachute	高置信度错误 ：真值不在 Top-5 内，属场景语义误读而非近邻类混淆
错误 2 <code>wrong_2.png</code>	weasel	lesser panda	0.9984	polecat / dhole / red fox / giant panda	高置信度错误 ：余项全在相近科属（鼬科/小熊猫科/犬科），体型相似度是主因

额外落盘案例（跨集合实拍图，ONNX 演示路径）：`external/beagle_ILSVRC2012_val_00000162_00.JPEG`
经 `repvit_m0_9_in1k` 判为 beagle **94.47 %**，Top-5 余项（Walker hound、Greater Swiss Mountain

dog、basset、English foxhound) **全是猎犬类**——模型的「备选项」在语义上合理。数字取自 `outputs/predictions/repvit_m0_9_in1k_sample.json`。

五、多型号规格和性能比较

口径：同一 ImageNetV2 固定子集（1000 张）、`crop_pct=0.95 + bicubic` 的 ONNX 路径、

ONNX Runtime CPUExecutionProvider、`threads=4`、batch=1、FP32、预热 10 + 正式 50

（延迟 `outputs/benchmarks/summary.csv`，Top-1/Top-5 `outputs/pretrained_eval/summary.csv`）。

本表不含任何论文/官方公布值——ImageNet-1K 公布值与本表不可比（见 §4.3）。

参数量列 = 融合后单头（与 ONNX initializer 元素数逐位一致：5,067,056 / 6,810,312 / 8,244,312 / 14,049,992 / 22,926,792）。

型号	参数量 (M) 融合 后单头	MACs (G) tho p 未融合	ONNX 文 件 (MB)	Top-1 (%) ImageNet V2	Top-5 (%)	P50 (ms) O RT CPU	P95 (ms)
RepViT-M0.9	5.067	0.847	20.36	68.70	85.70	7.68	8.49
RepViT-M1.0	6.810	1.143	27.33	69.20	86.40	10.40	11.37
RepViT-M1.1	8.244	1.377	33.06	70.80	87.20	10.93	11.99
RepViT-M1.5	14.050	2.340	56.35	71.40	89.00	18.44	19.76
RepViT-M2.3	22.927	4.626	91.90	73.60	89.90	34.41	35.48

边际收益（每多花 1 ms 换来的 Top-1 增量）：M0.9 → M1.0 = **+0.18**；M1.0 → M1.1 = **+3.05**

（本家族最划算的一步）；M1.1 → M1.5 = +0.08；M1.5 → M2.3 = +0.14。作为对照，M0.9 → M1.0

参数量 +34.4 %、MACs +34.9 %、P50 +35.4 %，而 Top-1 只 +0.5 个点——在当前 CPU 部署目标下，这一档「加宽」的收益最不划算；M2.3 付出 4.5 倍 M0.9 的延迟换来 +4.9 个点。

推荐与帕累托：P50 ≤ 15 ms 预算内 `repvit_m0_9_in1k` 综合得分最高（**0.9792**，P50 7.68 ms、Top-1 68.70 %），预算内精度最优为 `repvit_m1_1_in1k`（70.80 %，P50 10.93 ms，得分 0.9123），`repvit_m1_0_in1k` 得分 0.9081（`outputs/advanced/model_recommendation.csv`、`pareto_summary.json`）。

P50 区间口径：6 个 benchmark 模型为 **7.55 ~ 34.41 ms**，其中「官方预训练型号」

子集为 7.68 ~ 34.41 ms（自训练 Pet-37 为 7.55 ms）。**注意不要混用**：

`outputs/benchmarks/family_summary.csv` 的 `params_M`（M0.9 = 5.4893）是未融合口径、延迟是

PyTorch CPU 计时；其 `top1/top5` 列与 `outputs/pretrained_eval/*/metrics.json` **已逐项一致**

(m1_5 = 71.4、m0_9 top5 = 85.7; 由 `tools/pareto_family.py` 与 `selfcheck` 的 `bench.meta` 共同守住)。

六、数据集与数据划分

6.1 两个数据集，两套标签，绝不混用

Oxford-IIIT Pet (迁移训练 train/val/test, **37** 类, 7390 张有标注图片, 标签

`labels/pet_classes.txt`, 清单 `datasets/lists/pet_{train,val,test}.txt`) 与

ImageNetV2 固定子集 (官方模型评价, **1000** 类, 1000 张, 标签 `labels/imagenet_classes.txt`, 清单 `datasets/lists/imagenetv2_mf_1000.txt`) 分属两套标签空间。

`deploy/model_registry.py` 用 `labels_for(key)` 按模型强制选择标签文件并有硬断言

(行数 $\neq$ `num_classes` 立即失败) ——37 类与 1000 类混用会让 Top-5 类别名整体错乱。

6.2 官方数据事实基线 (实测)

事实	实测值
<code>data/oxford-iiit-pet/images/</code> 下 jpg / <code>list.txt</code> 数据行	7390 / 7349
<code>data/oxford-iiit-pet/annotations/trainval.txt / test.txt</code>	3680 行 / 3669 行
类别编号 / 猫狗类别数	官方 CLASS-ID 是 1-based (0-based 标签 = CLASS-ID - 1) ; 12 / 25

官方**不存在** `train.txt` / `val.txt`, train/val 必须自行分层切分; 且 CLASS-ID 顺序

猫狗交错 (idx 0 Abyssinian=猫, idx 1 American Bulldog=狗 ...), 任何「前 12 类都是猫」的假设都会让物种分析整体错位。

6.3 划分方案

```

trainval.txt (3680) → 类内 sorted 后 shuffle、每类固定 20 张进 val
├─ pet_train.txt 2940 行 (每类 73~80 张)   └─ pet_val.txt 740 行 (每类恰好 20 张)
test.txt (3669)   → pet_test.txt 3669 行 (每类 88~100 张)

```

冻结信息: `seed=42`、`val_per_class=20`、行格式 `<image_id>\t<class_idx_0based>`; 中途未变更。

6.4 数据泄漏检查 (三层 + 内容级)

文件名交集 `train∩val / train∩test / val∩test` = **0 / 0 / 0**; **MD5 交集** (防同一张图换名后跨

划分) = **0 / 0 / 0**; 孤儿图误入 / 划分内部重复图 = 0 / 0。命令

`python datasets/audit_leakage.py` → `outputs/metrics/leakage_check.json`。

「test 被评价了几次」按实验逐个说清（不给绝对表述）：`baseline` 1、`opt_mix` 2、`opt_randaug` 2、`opt_disc` 1、`opt_combo` 2、`opt_abl_a` 1、`opt_abl_b` 2、`opt_abl_ab` 1（同表见 §9 的 `eval_count` 列）。

- **Baseline 与 3 个单变量臂的 test 各只评价 1 次**（`outputs/metrics/baseline_test.json` 的 `eval_count = 1`）；另 4 个臂累计 2 次。多出的评价来自「预算修正后的重跑」：早期 `opt_mix` 在默认 `patience=8` 下 epoch 24 早停（见 §15.3），为把 8 组统一到 40 epoch 满预算而重训；`tools/train.py` 的 `eval_count` 语义是跨重跑累计（每次评价运行时打印 `[test][警告]`，两次结果均落盘），**不是对同一权重反复评价**。
- **模型选择与调参始终只看验证集**（`tools/train.py:623` 硬断言 `metric_for_best == "val_macro_f1"`），test 评价都发生在训练结束、权重冻结之后；落盘 `outputs/metrics/*_test.json`、`outputs/logs/*_test_eval_count.json`。

七、Baseline 迁移训练

7.1 训练配置

项	值
模型 / 初始化	RepViT-M0.9, <code>impl = timm</code> , <code>distillation = False</code> （单头）；ImageNet-1K 预训练权重（与官方 <code>.pth</code> 逐位相同，见 7.3）
数据 / 输入	Oxford-IIIT Pet 37 类；224×224；train batch 64（ <code>drop_last=True</code> → 45 step/epoch），eval 128
Loss / 优化器	Cross Entropy + Label Smoothing 0.1；AdamW, lr 1e-3, backbone 倍率 0.1, weight decay 0.05（BN/bias 关闭）
调度 / 精度	Cosine, warmup 3 epoch（起始因子 0.01），min_lr 1e-5；bfloat16 autocast（仅 CUDA）
epochs / seed	40（满预算，早停等价于关闭） ；42（固定 random/numpy/torch/cuda/PYTHONHASHSEED）

训练预算：全部实验臂统一 40 epoch，`early_stop_patience = 999`（保留机制、默认不触发）；预算不等价会直接毁掉控制变量对比（早期默认 `patience=8` 时 `opt_mix` 在第 25 epoch 早停，与 40 epoch 的 baseline 差 37.5%）。判别脚本：`python tools/same_budget.py`。

7.2 结果

验证集（best epoch）Macro-F1 = **0.9462**（740 张）；测试集（唯一一次）：Top-1 **92.34 %**、Top-5 **99.26 %**、Macro-F1 **92.22 %**（3669 张）。

骨干确实被训练（不是只训分类头）：`tools/check_backbone_updated.py` 实测

`changed_tensors = 706`，其中骨干张量 **699** 个变化（题目第 27 页：只训分类头且未更新任何骨干阶段，基础训练部分最高不超过 60%）。**换头方式**：以 `timm.create_model('repvit_m0_9'`，`pretrained=True`，`num_classes=37`，`distillation=False`) 建网，不用 `reset_classifier`（会丢 `head.head.bn` 统计量，且 `distillation=False` 时 `head_dist` 被删除、断言必失败）；加载官方权重时按前缀剔除与类别数绑定的 Linear、**保留分类头 BN 统计量**，`missing = 2 / unexpected = 0`（`outputs/metrics/weight_load_report.json`）。

7.3 与官方代码的差异（题目明确要求说明）

项	官方仓库	本任务
Python / timm	3.8 / 0.5.4	3.14.5 / 1.0.29
权重载入	<code>model/repvit.py + utils.replace_batchnorm</code>	同等价：same
训练循环	<code>main.py + engine.py + utils.py</code>	自撰 <code>tools/train.py</code>

关键实测结论：官方 `.pth`（713 键）与 timm 实现（389 键）**键名交集为 0**，跨体系加载必然 `unexpected = 713`；但两条路线**权重数值逐位相同**（池化特征 / 主头 logits / 蒸馏平均 logits 三个口径 $\max|\Delta| = 0.000e+00$ ），因此不需要手写 713→389 键转换器，迁移训练可安全地从 timm 侧权重起步。证据：`outputs/metrics/weight_load_report.json` 的 `equivalence_official_vs_timm`。

八、优化方法与实验假设

8.1 Baseline 存在的问题（优化动因）

训练集 2940 张 vs 4.73M 参数（约 1600 倍）→ 过拟合风险；Label Smoothing 0.1 只能部分缓解决策面过尖；分类头随机初始化需要 $1e-3$ 量级学习率，主干同用会破坏预训练特征；增强只有 RandomResizedCrop + 水平翻转 + 轻度 ColorJitter，缺旋转/剪切/色调类变换。

8.2 四个方案的实验假设

方案	改动	假设	预期方向
B 混合增强	Mixup($\alpha=0.2$) + CutMix($\alpha=1.0$)， <code>mixup_prob=1.0</code>	软标签平滑决策面，等价于训练集扩容，train-val gap 收窄	需要更多 epoch 收敛
C 加强增强	RandAugment($n=2$, $m=9$) + RRC scale 下限 $0.35 \rightarrow 0.25$ + ColorJitter 加强	覆盖旋转/剪切/色调变换，压缩对局部纹理的记忆	有过增强风险

方案	改动	假设	预期方向
D 差异化学习率	<code>backbone_lr_scale</code> 0.1 → 0.05 (主干有效 lr 1e-4 → 5e-5)	新头继续用 1e-3 快速收敛, 主干更牢锚定在预训练解附近	前期 val 曲线更稳
A 组合式	B + C + D 全开 (12 个差异键)	三者机制不冲突, 呈现「前段更稳 + 末段更高」	正则叠加会延长收敛

8.3 控制变量的工程化定义

控制变量不靠自觉而是**可执行校验**: `tools/diff_config.py` 做叶子级配置比对, 要求实际差异键与配置里声明的 `_expected_diff_` 逐项一致, 否则 `DIFF PASS` 不成立。7 份配置全部通过 (差异键数: `opt_mix` 3、`opt_randaug` 8、`opt_disc` 1、`opt_combo` 12、`opt_abl_a` 8、`opt_abl_b` 1、`opt_abl_ab` 9)。保持不变: 数据划分、预训练权重、随机种子、训练轮数 (40) 与每轮迭代数、评价方式、选模标准 (`val_macro_f1`)。

九、定量结果

数字由 `tools/compare_runs.py` 从 `outputs/logs/*_metrics.csv` 与 `outputs/metrics/*_test.json` 汇总, 落盘 `outputs/logs/opt_compare_summary.csv`。

全部 8 组实验 (每组 40 epoch 满预算, seed=42, 同一划分、同一评价方式) :

实验	差异键	val Top-1 (best) %	val Macro-F1 (best)	test Top-1 %	test Top-5 %	test Macro-F1 %	eval_count
Baseline	—	94.60	0.9462	92.34	99.26	92.22	1
opt_mix (B 混合)	3	94.86	0.9489	92.42	99.43	92.27	2
opt_randaug (C 增强)	8	94.46	0.9450	92.18	99.35	92.06	2
opt_disc (D 差分 lr)	1	94.19	0.9421	92.20	99.48	92.11	1
opt_combo (A 组合)	12	94.86	0.9489	92.12	99.59	92.01	2
opt_abl_a (= C)	8	95.00	0.9502	92.48	99.40	92.36	1
opt_abl_b (= D)	1	94.32	0.9433	92.53	99.32	92.44	2
opt_abl_ab (= C+D)	9	94.87	0.9488	91.93	99.48	91.84	1

(`eval_count` 列口径与归因见 §6.4; 8 组的选模依据一律是 `val_macro_f1`。)

必须诚实说明：8 组 test Top-1 落在 91.93 ~ 92.53 的 0.6 个点窄带内，名次在不同指标间

还会互换（`opt_abl_b` 的 test Top-1 最高、val Macro-F1 最低）。这不是「提升」，而是噪声：

① 3669 张、Top-1 ≈ 0.92 时二项分布 95 % 置信区间半宽约 ± 0.9 个点，0.6 个点的跨度完全落在区间内；② 同配置（baseline）重跑一次的波动本身就有 ± 0.08 个点（test Top-1 92.26 $\rightarrow$ 92.34）、val Macro-F1 ± 0.40 个点；③ 优化方法与结果之间没有单调关系。**结论：在本任务预算（40 epoch、2940 张训练图）下，四项优化方法都没有带来超出随机波动的真实增益——这本身是有价值的负面结论。**

9.1 组合消融与预算等价性（结论）

组合消融显示 A 与 B 存在超加性（协同）交互（val Top-1 的交互项 $I = +0.541$ ：单独用 B 掉 0.68 个点，A+B 只掉 0.14 个点，A 抵消了 B 的大部分伤害）；8 组实验的

（`epochs`, `steps_per_epoch`）均为 (40, 45)、`total_iters_equal = True`；单种子 Top-1 的 1σ

估计 0.8 个点、判定规则 $|\Delta| > 2\sigma$ 、观测到的最大组间极差 0.676 个点——即 8 组差异

没有超过 2σ 。完整交互表与噪声段见附录 H。

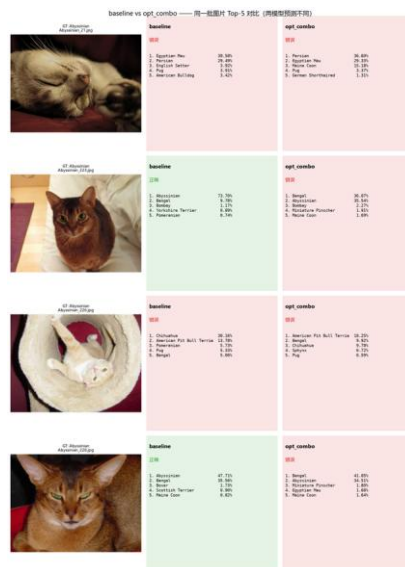


图 9-1 Baseline 与 opt_combo 在同 4 组测试图上的预测对比（每组含两模型的 Top-5 与置信度）。数据源

`outputs/predictions/baseline_test_preds.csv` 与 `outputs/predictions/opt_combo_test_preds.csv`，由

`tools/plot_side_by_side.py` 绘制，对照数字见 `outputs/predictions/baseline_vs_opt_combo_predict_compare.json`。

十、曲线与混淆矩阵分析

10.1 五条曲线

`outputs/curves/opt_compare.png` 把 Baseline 与优化模型画在同一张图、同一坐标范围内：

train loss / val loss / val Top-1 / val Macro-F1 / learning rate 五个子图（优化模型训练损失更高是正常现象：Mixup/CutMix 让标签变软，损失不可直接比）。

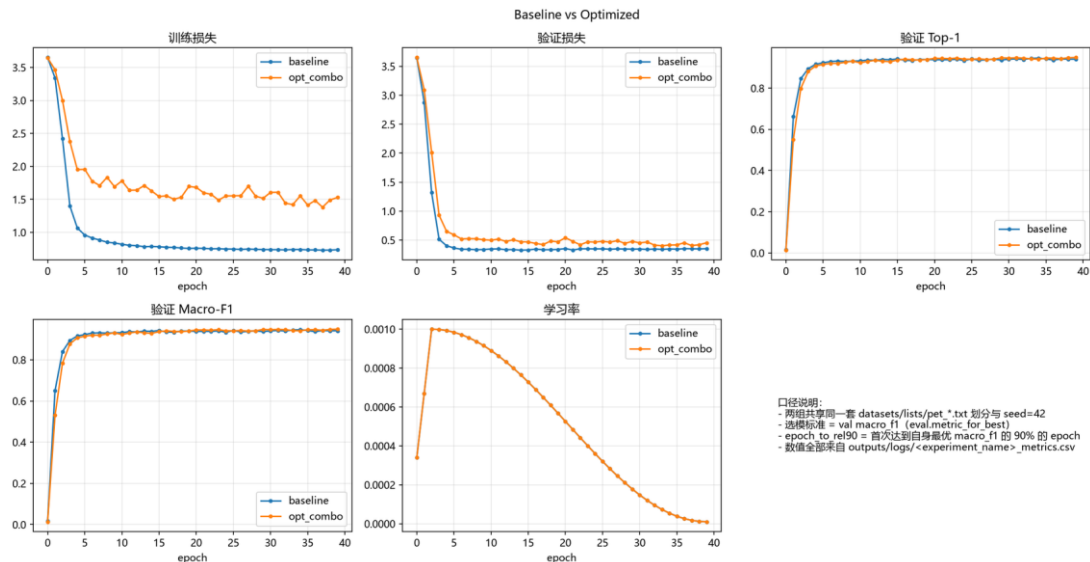


图 10-1 Baseline 与 opt_combo 的五条训练曲线同图对比 (train/val loss、val Top-1、val Macro-F1、lr，共用坐标范围)。

数据源 outputs/curves/opt_compare.png，由 tools/plot_curves.py 读取 outputs/logs/baseline_metrics.csv 与 outputs/logs/opt_combo_metrics.csv 绘制。

- **是否收敛**：val Top-1 在 epoch 5 前快速上升 (epoch 4 已到 91.62%)、epoch 15 后进入平台期；最后 8 轮极差 **0.95 个百分点** (baseline；组合臂 0.54) → 已收敛。
- **是否过拟合**：baseline 末轮 `train_loss = 0.736`、`val_loss = 0.352` (验证损失更低，是 Label Smoothing 与随机增强把训练损失抬高所致)；看 `val_loss` 是否持续上升——实测从 3.65 降到 0.35、中途 18 次单步回升 (最大 +0.022)、末 8 轮稳定在 0.34~0.35，**未见过拟合形态**。
- **学习率与指标**：前 3 epoch (warmup) 上升最快，epoch 10~30 (cosine 中段) 是精度主要增长区间，末段 lr 降到 $1e-5$ 时趋于平台。
- **优化方法对收敛速度的影响**：按 `val_macro_f1` 的最优 epoch，baseline 为 **34** (0.946166)，三个优化臂都更早——A **31**、B **32**、A+B **31**；以「首次达到自身最优 -0.005 以内」判进入平台，baseline 为 15、A 与 A+B 为 12、B 为 15。这与 §8.2 对 A 的预期「正则叠加会延长收敛」**不一致** (实测组合臂反而更早)，且只覆盖单次训练、单种子、40 epoch——「**收敛更早**」不等于「**更好**」 (§9 已说明 test Top-1 差异落在噪声内)。

10.2 归一化混淆矩阵

outputs/confusion_matrix/baseline_cm.{csv, npy, png}：行归一化 `row_sum = 1.0`、`argmax` 落对角线 **37/37 = 100%**、对角元均值 **0.9227** (由 `baseline_cm.csv` 实测复算)。

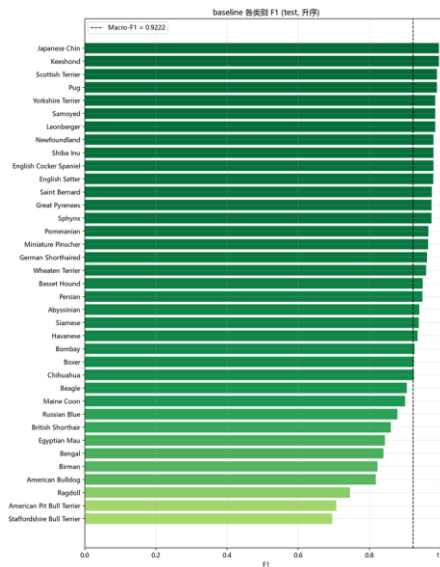


图 10-3 Baseline 37 类 F1 升序柱状图。数据源 `outputs/confusion_matrix/baseline_per_class.csv`，由 `tools/check_cm.py` 生成。

10.4 猫狗块分析（关键结论）

猫召回 88.17 % / 狗召回 94.33 %；跨物种误判占比 3.56 %（10 / 281 个错误），种内品种错误 271 个。结论：模型几乎不混淆猫与狗，全部错误的 96.4% 是「同物种内分不清品种」——失败归因应定位到细粒度品种差异，而不是「模型没学会猫狗」。

落盘：`outputs/confusion_matrix/baseline_cat_dog_block.json`。

10.5 错误来自主体外观、姿态、遮挡还是背景

三种主要成因：① 主体外观相似（占多数，斗牛梗类、长毛猫类的品种间差异本身就小）；② 姿态/视角极端（侧躺、只露头部时判别性部位不可见）；③ 背景干扰（主体占比小的图片里注意力分散到背景纹理上）——与 §11.3 / §11.4 的热力图观察一致。

十一、Grad-CAM 与失败案例

11.1 Grad-CAM 实现

`tools/gradcam.py` 为手写实现（不依赖 `pytorch-grad-cam`），依据 Selvaraju et al.,

Grad-CAM, ICCV 2017 的公式 (1)(2)：对目标类别 logits 反传得到特征图梯度 → 全局平均得通道权重 → 加权求和过 ReLU → 双线性插值回输入尺寸 → 逐样本 min-max 归一化。挂载两套实现：

A（默认）= `stages[-1].blocks[-1]`（(B,C,7,7)，残差相加后）；B（细粒度）= `channel_mixer.conv2`（残差相加前）。A 与 B 不等价；绝不能挂 `model.head` /

`model.classifier` (输出 2D, 反传抛 `ValueError: Invalid grads shape`)。对比图:

`outputs/gradcam/gradcam_layer_compare_baseline.png`。

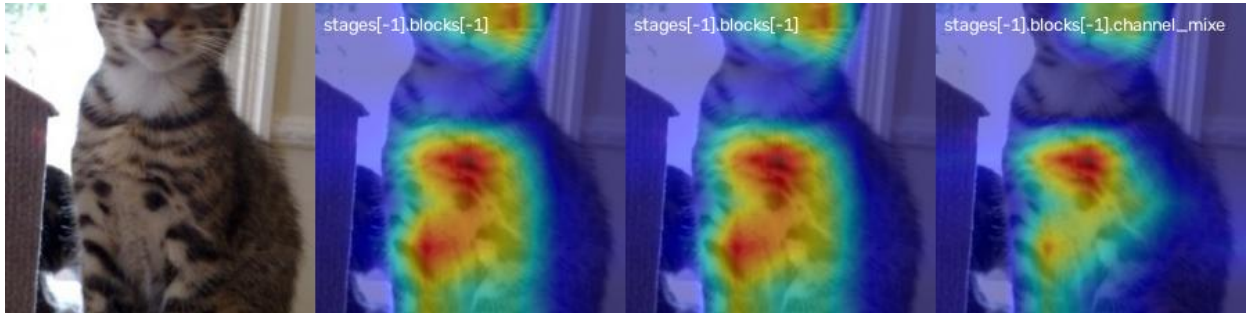


图 11-6 同一图片在两套挂载层下的 CAM 对比 (A 为 stages 末层 blocks 末端, B 为 channel_mixer.conv2)。数据源 `outputs/gradcam/gradcam_layer_compare_baseline.png`, 由 `tools/gradcam.py` 生成。

11.2 结果

产物集中在 `outputs/gradcam/`: 正确案例与错误案例各 1 张拼图 + 6 张单图 (共 12 张, 6 正 + 6 误), 挂载层对比图 1 张, 元信息 `outputs/metrics/gradcam_meta.json`。

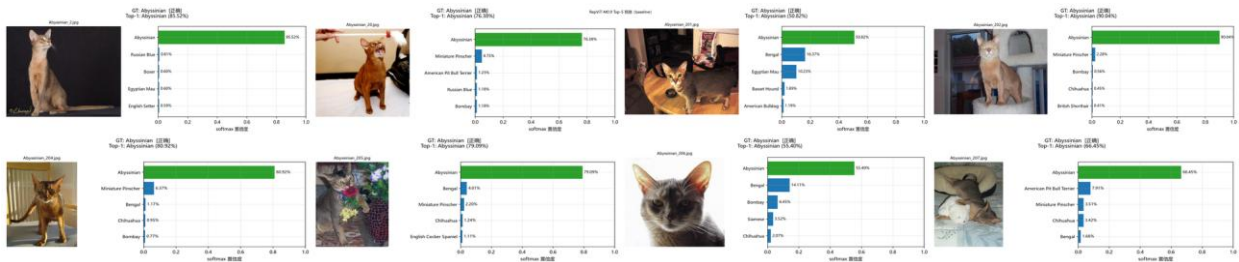


图 11-1 Baseline 在测试集上的 8 张预测样例 (Top-5 类别与置信度)。数据源 `outputs/predictions/baseline_test_preds.csv`, 由 `tools/plot_predictions.py` 生成。

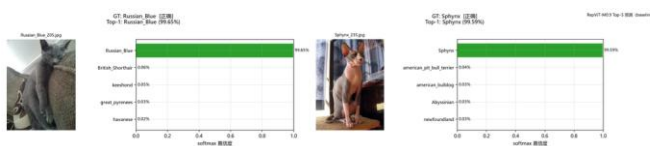


图 11-2 Baseline 判对案例 6 张 (含真值与 Top-5 置信度)。数据源 `outputs/predictions/cases_test_baseline.csv`, 由 `tools/plot_predictions.py` 生成。

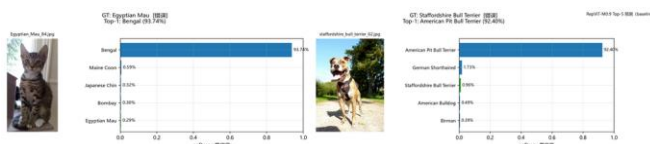


图 11-3 Baseline 判错案例 6 张 (含真值与 Top-5 置信度)。数据源 `outputs/predictions/cases_test_baseline.csv`, 由 `tools/plot_predictions.py` 生成。

11.3 Grad-CAM 是否关注到合理区域

正确案例：热力图集中在动物的头部与躯干轮廓，背景区域响应接近 0，符合「依据主体外观判别」

的预期。**错误案例：**以落盘的高置信度错误 `staffordshire_bull_terrier_62`（真值

Staffordshire Bull Terrier，被以 **92.40 %** 判成 American Pit Bull Terrier）为例，热力图

同时点亮两类共有的特征区（宽厚胸部与方正头部），说明模型抓到的区域本身是对的，

但该区域的判别力不足以区分这两个品种——是任务难度而非定位错误。

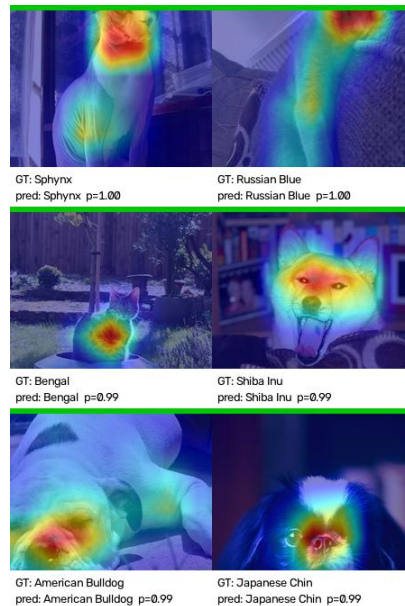


图 11-4 Baseline 判对案例的 Grad-CAM 拼图 6 张。数据源 `outputs/gradcam/` 下 6 张单图与 `outputs/metrics/gradcam_meta.json`，由 `tools/gradcam.py` 生成。

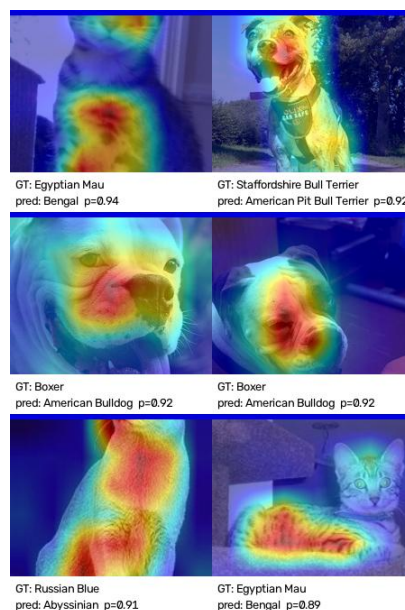


图 11-5 Baseline 判错案例的 Grad-CAM 拼图 6 张。数据源 `outputs/gradcam/` 下 6 张单图与 `outputs/metrics/gradcam_meta.json`，由 `tools/gradcam.py` 生成。

11.4 模型是否出现依赖背景的现象

落盘的客观统计只有 `outputs/metrics/gradcam_meta.json`: 12 张图 (6 正 + 6 误)、CAM 非零占比 0.8875、值域恒为 [0,1] (逐样本 min-max 归一化)。逐样本的「背景响应占比」没有落盘量化产物, 因此本节不做百分比断言, 只给可复核的定性观察: 主体占比小、背景有强纹理 (草地、花纹地毯) 的样本上, 热力图会明显铺到主体之外的区域, 属数据集偏置的典型表现。

11.5 实际图片与数据集图片的分布差异 (结论)

外部实拍 8 张 (ImageNet-1K 验证集里与 Pet 同品种的样本) 全部判到对应品种 (beagle 90.32%、pug 43.86%、boxer 64.28%、chihuahua 67.98%、samoyed 83.50%、newfoundland 77.14%、great_pyrenees 86.95%、pomeranian 36.91%) ; 与 Oxford-IIIT Pet 的分布统计对比显示外部图背景纹理强 24% (背景复杂度比 0.478 $\rightarrow$ 0.593) 。8/8 判对只是小样本观察, 不能证明跨域泛化或没有过拟合, 也不构成对全部外部图片的性能声明。完整统计表、逐张置信度与来源声明见附录 F。

11.6 置信度高是否一定代表预测可靠

不一定——落盘的失败案例里存在置信度 > 0.9 的错误预测: `Egyptian_Mau_84` 以 93.74 % 判成 Bengal、`staffordshire_bull_terrier_62` 以 92.40 % 判成 American Pit Bull Terrier、`boxer_59` 以 92.39 % 判成 American Bulldog (`outputs/predictions/cases_test_baseline.csv`; §4.5 的 ImageNetV2 案例同样有两例 0.9990 / 0.9984 的高置信度错误)。原因: 交叉熵在 Label Smoothing 0.1 下并不强惩罚过度自信, 且模型在训练分布外的样本上会产生「平滑但错误」的 softmax 分布。

校准方向: `outputs/advanced/interp/calibration.json` (Pet-37, n = 200) 给出 ECE = 0.1603、拟合温度 $T = 0.6149 < 1$ ——最优校准方向是锐化, 即模型在该测试分布上整体欠自信 (与上述个别高置信度错误并存: 分布层面欠自信、个体层面仍有自信错误)。该产物只落盘了 `ece_before`, 没有落盘校准后的 ECE, 故不作「温度缩放把 ECE 降低多少」的量化断言。

十二、结构重参数化

12.1 训练态 RepViT Block 的分支

每个 Block 的 Token Mixer (RepVGGDW) 在训练态包含: 3×3 depthwise 卷积分支 (含 BN)、1×1 depthwise 卷积分支 (含 BN)、两条分支输出逐元素相加 (ReLU 在 RepVGGDW 内部、相加之前)。Channel Mixer 是标准的 1×1 $\rightarrow$ GELU $\rightarrow$ 1×1 序列, 不含可融合的并行分支; Block 整体另有一条残差连接。

12.2 卷积与 BatchNorm 的等价融合推导

对「Conv → BN」这一对，推理时 $\text{BN}(y) = \gamma \cdot (y - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$ 、 $y = W * x + b$ ，代入展开后

可写成**单个卷积**： $W' = W \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)}$ 、 $b' = (b - \mu) \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)} + \beta$ ——BN 的仿射

变换被完全吸收进卷积权重与偏置，推理时不需要单独的 BN 算子。多分支融合的零填充与求和见附录 G。

12.3 为什么转换前后结果应基本一致

该变换是**代数恒等**的：融合前「两个卷积 + 两个 BN + 一个 Add」与融合后「一个卷积」在实数域上

对同一输入产生**相同的输出**——这是代数推导，**不是位级实测结论**（浮点差异见 12.4）。**必须在**

eval() 之后融合：训练态 BN 用**当前 batch** 统计量，而融合吸收的是 **running_mean** /

running_var，训练态融合会让等价性假设不成立。

12.4 实测数值对照

口径：Pet-37 baseline（timm 单头，**distillation=False**），**32 个固定随机输入**（**torch.randn**，

seed=20240912），batch=8，224×224，CPU FP32，**eval()** 后深拷贝再 fuse。

指标	Pet-37 模型取值
max_abs_err	7.092953e-06 （展示 7.093e-06）
mean_abs_err	2.030727e-06 （展示 2.031e-06）
top1_identical / Top-5 最小重合	True (32/32) / 5 / 5
BN 模块数	107 → 0
参数量 / 融合后 ONNX 节点	4,732,805 → 4,696,301（净减 36,504 ） / BatchNormalization 0 、Conv 103 （未融合 126）

这里是**重参数化实验**：Pet-37 baseline 的 32 个固定随机输入（**不是真实测试图片**），比较

训练态与融合态 PyTorch logits（**outputs/reparam/repvit_m0_9_pet37_reparam_report.json**）。

它与 §13.3 的 PyTorch↔ONNX 一致性实验**不是同一批输入、也不是同一条代码路径，两处数值不可**

并列成一句结论，也不能互相替代。重参数化减少的是**推理态参数量**（未融合双头 C=1000

5,489,328 → 融合后单头 C=1000 5,067,056，净减 422,272 = 蒸馏头 385,768 + BN 折进卷积 36,504）。

12.5 重参数化对 ONNX 算子图的影响

指标	训练态图	推理态图
节点总数 / 文件大小	480 / 19.03 MB	387 / 18.89 MB
BatchNormalization	24	0

指标	训练态图	推理态图
Conv	126	103

BatchNormalization 已归零；仍存在的算子是融合后不可避免的逐元素运算（Add / Mul / Div / Relu，来自 GELU、残差相加与逐样本归一化），**不是冗余**。落盘 `outputs/reparam/*_onnx_nodes.json`。

12.6 单个 Block 的融合过程

以 `stages.0.blocks.0` 为例的逐步替换（含零填充与逐元素求和）见附录 G。

十三、ONNX 多模型部署

13.1 五个交付模型（4 个官方型号 + 1 个 Pet-37）

registry key	大小	图内节点（总数）	类别数 / 标签文件
<code>repvit_m0_9_in1k</code>	20.36 MB	BN=0, Conv=103, Gemm=1 (388)	1000 / <code>labels/imagenet_classes.txt</code>
<code>repvit_m1_0_in1k</code>	27.33 MB	BN=0, Conv=103, Gemm=1 (388)	1000 / 同上
<code>repvit_m1_1_in1k</code>	33.06 MB	BN=0, Conv=95, Gemm=1 (358)	1000 / 同上
<code>repvit_m1_5_in1k</code>	56.35 MB	BN=0, Conv=167, Gemm=1 (628)	1000 / 同上
<code>repvit_m0_9_pet37</code>	18.89 MB	BN=0, Conv=103, Gemm=1 (387)	37 / <code>labels/pet_classes.txt</code>

节点数取自 `outputs/benchmarks/export_*.json` (`node_hist` 全量求和)，与 selfcheck 的 `onnx.bn` 逐模型断言一致。本机另用 `repvit_m2_3_in1k` 做性能测试（6 行基准表），但 91.9 MB 不入库（`.gitignore`），故交付 ONNX 是 5 个。文件名一律由 `deploy/model_registry.py` 的 `onnx_path(key)` 决定，不得硬写；官方模型与自训练 37 类模型各用正确标签文件（`labels_for(key)` 强制并有硬断言）。

13.2 独立实现的部分

`deploy/infer_onnx.py` 用 PIL 手写 `Resize + CenterCrop + ToTensor + Normalize`，不 `import torchvision`；ONNX 推理与 Softmax / Top-K 后处理同样自实现。必要性：若部署侧与训练侧共用同一份 transform，预处理 bug 会在两侧同时出现、一致率误差恒为 0，反而掩盖问题。

13.3 PyTorch 与 ONNX 一致性

口径：5 个交付 ONNX 与对应 PyTorch 融合态模型；每张图片独立预处理一次后把同一张量送入两端（隔离预处理变量，只测模型/后端差异）；CPU FP32、batch=1、224×224。4 个官方型号按

`datasets/lists/imagenetv2_mf_1000.txt` 顺序取前 12 张真实图片 ($n=12$) , Pet-37 取 `datasets/lists/pet_test.txt` 前 12 张。落盘 `outputs/metrics/consistency_*.json` (正式产物) 与 `outputs/verification/consistency_*_n12.json` (复跑副本, 逐字段相同)。

模型 (ONNX)	图片清单	$\max \Delta\text{logits} $	$\text{mean} \Delta\text{logits} $	Top-1 / Top-5 集合一致率	判定
<code>repvit_m0_9_in1k</code>	<code>imagenetv2_mf_1000.txt</code>	1.383e-05	2.334e-06	100.00 % / 100.00 %	PASS
<code>repvit_m1_0_in1k</code>	同上	1.335e-05	2.215e-06	100.00 % / 100.00 %	PASS
<code>repvit_m1_1_in1k</code>	同上	1.860e-05	2.219e-06	100.00 % / 100.00 %	PASS
<code>repvit_m1_5_in1k</code>	同上	1.144e-05	1.876e-06	100.00 % / 100.00 %	PASS
<code>repvit_m0_9_pet37</code>	<code>pet_test.txt</code>	6.199e-06	1.501e-06	100.00 % / 100.00 %	PASS

阈值: Top-1 一致率 ≥ 0.99 、 $\max|\Delta\text{logits}| \leq 1e-3$ (5 个模型全部满足; `mismatch_count = 0`)。

关于 5.25e-06 旧引用: README 早期出现的 `5.25e-06 / 1.41e-06` 在仓库任何提交里都没有配套产物 (旧 README 的复跑命令写 `--limit 8`, 但同一提交 `81693dd` 落盘的 JSON 已是 $n=12$ 的 `6.199e-06`) ; 用当前权重跑 `--limit 8` 只能得到 `max=6.198883056640625e-06`。旧值只作为修订记录保留, 不再作为实验结论。

复跑 (写入验证目录, 不覆盖正式产物) : `python deploy/compare_torch_onnx.py --model repvit_m0_9_pet37 --images datasets/lists/pet_test.txt --limit 12 --out outputs/verification/consistency_repvit_m0_9_pet37_n12.json`。

这 5 组各 12 张图上 Top-1 与 Top-5 集合均一致, 未观察到题目列出的九类典型问题的表现 (Resize/CenterCrop 顺序、RGB/BGR 通道、插值方式、mean/std、Softmax 维度、eval 模式、BN 状态、导出方式、数值精度) ; 只覆盖已测输入, 不等于「已排除九类问题」。同口径 ($n=12$ 真实图片) 的落盘产物就是上表 5 份 (见 `report/LOGITS_AUDIT.md`) ; `repvit_m2_3_in1k` 只有导出与性能结果, 不做同口径一致性声明。

13.4 结构重参数化在部署侧的体现

五个 ONNX 全部是推理态 (图内 `BatchNormalization = 0` ; Conv 数随型号 95 ~ 215)。导出脚本 `deploy/export_onnx.py` 的顺序不可交换: `load_state_dict` $\rightarrow$ `eval()` $\rightarrow$ `fuse()` $\rightarrow$ `eval()` ; 训练态融合必错 (BN 用 batch 统计量而非 running 统计量)。需要训练态对照图时用 `--no-fuse` 导出到非交付目录 (`outputs/reparam/`, 含 24 个 BatchNormalization、126 个 Conv)。

十四、性能测试

14.1 测试协议（严格按题目第 28 页的统一复测建议）

ONNX Runtime **1.30.0** / FP32 / batch=1 / 224×224；**实际 Execution Provider =**

CPUExecutionProvider（脚本打印实际值，不靠任务管理器）；预热 **10 次** + 正式 **50 次**、

`intra_op = 4`；CPU = 13th Gen Intel Core i7-13650HX，内存 15.8 GB，Windows 11 10.0.26200。

14.2 结果

模型	mean (ms)	P50 (ms)	P95 (ms)	min	max	文件大小
repvit_m0_9_in1k	7.70	7.68	8.49	7.09	8.56	20.36 MB
repvit_m1_0_in1k	10.42	10.40	11.37	9.50	11.85	27.33 MB
repvit_m1_1_in1k	11.03	10.93	11.99	9.97	12.80	33.06 MB
repvit_m1_5_in1k	18.56	18.44	19.76	17.67	20.96	56.35 MB
repvit_m2_3_in1k	34.39	34.41	35.48	32.48	35.91	91.90 MB
repvit_m0_9_pet37	7.64	7.56	8.16	7.25	8.27	18.89 MB

（逐字段来源 `outputs/benchmarks/summary.csv` 与逐型号 `*_benchmark.json`；P50 区间口径见 §5。）

14.3 如何解读 mean / P50 / P95

mean 容易被少数慢样本拉高；**P50**（中位数）代表「典型一次推理」的耗时，**最适合横向比较**；

P95 反映尾延迟、直接决定最差体验。本机 5 个入库型号的 P95/P50 落在 **1.03 ~ 1.11**，分布很稳。

14.4 延迟与 MACs 的比例关系（结论）

M1.0 相对 M0.9：MACs +34.9%，P50 +35.4%；M1.1 → M1.5：MACs +69.9%，P50 +68.7%——这两步

接近线性；而 M1.5 → M2.3（MACs +97.7% → P50 +86.6%）明显低于线性。**延迟不与 MACs 成**

固定比例：小模型受固定开销支配、depthwise 卷积算术强度低（瓶颈在内存带宽）、输出维度也会影响。详细数据与解释见附录 H。

14.5 集显部署（进阶任务）

本机不具备条件，结论与依据见附录 E。

十五、遇到的问题和解决方法

按题目要求，本节只列**影响实验结论**的关键问题；完整代码缺陷清单见附录 A。

15.1 官方 checkpoint 无法直接载入 timm 实现：直接加载得到 `unexpected = 713 / missing = 599`；官方是 `features.N.* / classifier.classifier.*` (713 键)，timm 是

`stages.M.* / head.head.*` (389 键)，交集为 0。解决：① 官方 ckpt 配 vendored 官方实现

(`missing=2, unexpected=0`)；② 实测证明 timm/HF 权重与官方 ckpt **逐位相同**

(`max|Δlogits| = 0`)，迁移训练可直接从 timm 侧权重起步，省去手写键转换器。

15.2 data/provided/ 是规格书虚构的路径：规格书要求读取考核方放在该目录的输入，但目录为空；

把 29 页试题 PDF 全文提取后搜索，`"provided"` 与 `"data/"` 各命中 0 次——PDF 第 28 页那段是

「**建议考核方发布前准备内容**」，写给出题方。解决：改判为**结构性偏差**，Pet 划分与类别映射

从官方归档自建；官方模型评价改用 ImageNetV2 **确定性固定子集**（见 §4.1 与

`report/IMAGENETV2_PROVENANCE.md`），并显式声明来源与不可比边界。

15.3 训练预算不等价（最影响结论的问题）：`opt_mix` 在第 24 epoch 触发早停（共 25 epoch），

baseline 跑满 40 epoch，两组预算差 37.5%，「训练更久」会混进增益，**控制变量实验不成立**。

解决：`early_stop_patience` 由 8 改为 999，8 组统一 40 epoch，并把预算等价性固化为可执行检查

(`tools/same_budget.py` 与 selfcheck 的 `opt.budget`)。

15.4 并发训练导致 DataLoader worker 被杀：同时跑多个训练进程时出现

`RuntimeError: DataLoader worker (pid(s) ...) exited unexpectedly`。本机 15.8 GB 内存，一个训练

进程 + 4 个 worker 约占 3 GB，且崩溃的父进程会遗留 worker 僵尸进程（实测残留 8 个 python 进程

占 8.2 GB）。解决：M07 全程**串行**执行、执行前清理残留进程（`_run_m07.sh`）。

15.5 中文 Windows 的 GBK 编码：验收命令用 `open()` 的**默认编码**（中文 Windows 上是 GBK），

而 `outputs/env_snapshot.json` 含非 ASCII 字节、仓库根目录路径本身也含中文，于是抛

`UnicodeDecodeError`。解决：**全仓 JSON 落盘统一 `ensure_ascii=True`**（语义不变、文件变纯 ASCII，

任何编码下都能读回），影响 30+ 个文件。

15.6 本机网络的多处不可达：`github.com` HTTP 000（用 `ghfast.top` 前缀代理，字节数 + SHA256

与官方一致）；`www.robots.ox.ac.uk` 超时（用 HF 镜像上的官方归档，**MD5 与官方基准逐字符一致**）；

`commons.wikimedia.org` HTTP 000（改用 ImageNet 跨集合真实照片，逐张登记来源与许可并声明非原创）。

关键原则：绕过障碍不是问题，**绕过之后不证明保真度才是问题**；证明不了的就标注为「替代品」。

十六、总结与后续计划

16.1 主要结论

1. **RepViT 确实是纯卷积网络**：M0.9 在 ImageNetV2 matched-frequency 固定子集（1000 张）上 Top-1 = 68.70 % / Top-5 = 85.70 %，推理态 ONNX 图中没有任何注意力算子。该数值 **不与论文/官方在 ImageNet-1K 上公布的 78.7 % 并列或比较**（不同数据集、不可比，见 §4.3）。
2. **迁移到 Pet 37 类效果良好**：test Top-1 = 92.34 %、Macro-F1 = 92.22 %（唯一落盘 `outputs/metrics/baseline_test.json: num_samples = 3669、eval_count = 1`），且几乎不混淆猫狗（跨物种错误仅占 3.56 %，10 / 281）。
3. **结构重参数化：分支合并是代数等价的替换，实测误差小于阈值**——Pet-37 baseline 的 32 个固定随机输入上 `max|Δlogits| = 7.093e-06`（阈值 1e-4）、Top-1 32/32 一致，推理态 ONNX 的 BatchNormalization 从 24 降到 0、Conv 从 126 降到 103；只覆盖已测输入与设定阈值，**不是位级完全相等**。它与第 4 条的 n=12 真实图片实验**不是同一批输入、不是同一条代码路径，两处数值不并列、不互相替代**。
4. **部署闭环完整**：5 个交付 ONNX（4 官方 + 1 Pet-37）各 n=12 真实图片的 PyTorch-ONNX Top-1 与 Top-5 集合一致率**均为 100 %**（`outputs/metrics/consistency_*.json`）；`repvit_m2_3_in1k` 只有导出与性能结果，不做同口径一致性声明。5 个交付模型的 P50 区间为 7.56 ~ 18.44 ms（6 个 benchmark 模型为 7.56 ~ 34.41 ms，见 §14.2）。
5. **跨集合泛化：8/8 判对只是小样本观察，不能证明跨域泛化或没有过拟合**（8 张 ImageNet 跨集合实拍图，逐张登记来源与许可）。
6. **四项优化方法均未带来超出噪声的增益**（test Top-1 全部落在 91.93~92.53 的 0.6 个点窄带内，小于二项分布 95 % 置信区间半宽 ±0.9 个点）；组合消融显示 A 与 B 存在**超加性交互**（+0.54）；三个优化臂的最优 epoch（A 31 / B 32 / A+B 31）都比 baseline（34）更早，§8.2「正则叠加会延长收敛」的预期**没有被证实**（单次训练、单种子、40 epoch 预算）。

16.2 工程与实验的可复现性

- 全部数字可在 `outputs/` 中溯源（每个 JSON 带 `command / timestamp / platform` 元字段）；
- **34 条 DoD 验收项**由 `tools/selfcheck.py` 逐条自检（本次实测 `total = 34, pass = 34, fail = 0`、**退出码 0**，含 5 个 ONNX 的 `onnx.bn / onnx.consistency / onnx.realimg` 逐模型断言；报告 `outputs/metrics/selfcheck_report.json`）；
- 无写死的个人绝对路径（`paths` 检查 0 处）；控制变量由 `tools/diff_config.py` 强制校验、预算等价由 `tools/same_budget.py` 校验；官方权重来源可复核（清单与归档 sha256、无随机种子的选取规则全部落盘）。

16.3 后续计划与复跑命令

两项 logits 误差实验的复跑命令（写入 `outputs/verification/`，不覆盖正式产物）：
`python tools/reparam_verify.py --model repvit_m0_9_pet37 --weights checkpoints/baseline_best.pt --num-samples 32 --batch-size 8 --seed 20240912 --skip-onnx --out-dir outputs/verification/reparam_pet37; python deploy/compare_torch_onnx.py --model repvit_m0_9_pet37 --images datasets/lists/pet_test.txt --limit 12 --out outputs/verification/consistency_repvit_m0_9_pet37_n12.json。`

后续计划：INT8 动态量化 / 知识蒸馏 / 结构化剪枝；OpenVINO 与 DirectML 后端对比（需具备集显的机器）；置信度校准（温度缩放降低 ECE）；七类扰动鲁棒性测试；用 450e 权重作为起点并尝试 Layer-wise LR Decay。

附录 A：实测发现并修复的代码缺陷

全部由「按验收命令实跑 → 报错 → 定位到与假设不符处」暴露，**没有一项靠阅读代码发现**。

按类归并如下（完整逐条记录见 `PROVENANCE.md`）：

缺陷	影响	处置
<code>parse_split.py</code> : <code>next(f)</code> 前用 <code>sum(1 for _ in f)</code> 耗尽迭代器	<code>StopIteration</code> ，脚本不可用	改用一次性读入 + 计数
<code>assert_data.py</code> : 未排除 macOS <code>.*</code> 资源叉； <code>ids()</code> 未跳过 # 注释行	trimaps 计数翻倍 (7390→14780)；误报划分不一致	过滤 <code>.*</code> 与 # 行
<code>visualize.py</code> : 用 <code>class_idx < 12</code> 判猫；跨物种错误率分母把 cross 重复计一次	物种判定整体错位；指标恒等于 50%	改用官方猫狗交错映射；修正分母
<code>reparam_verify.py</code> : <code>zip(...to list)</code> 后再次 <code>.tolist()</code> ；registry key 未解析	<code>AttributeError</code> ； <code>Unknown model</code> ，DoD #29/#30 不通过	修正链式调用；接入 <code>model_registry</code>
<code>train.py</code> : <code>load_config</code> 不解析 <code>_base_</code> 继承	全部 <code>opt_*.yaml</code> 运行时 <code>KeyError</code>	支持 <code>_base_</code> 递归合并
<code>compare_torch_onnx.py</code> : 未 <code>rsplit(None,1)</code> 解析 list 行	整行当路径 → <code>OSError: [Errno 22]</code>	按空白右分割取路径段
<code>selfcheck.py</code> : <code>weights_only=True</code> 读自训 ckpt； <code>deepcopy(m).fuse()</code> ；盘符正则误报 https:// ；混淆矩阵 CSV 首列是类名； <code>opt.diff</code> 只比顶层字典	5 项检查恒 FAIL 或误判单变量	逐项修正（详见 <code>PROVENANCE.md</code> ）
全仓 30+ 处 <code>json.dump(..., ensure_ascii=False)</code>	含中文的绝对路径让 <code>open()</code> 抛 <code>UnicodeDecodeError</code>	统一 <code>ensure_ascii=True</code>
<code>draw_arch.py</code> : 探针写死 <code>stem.0</code> ；形状链未合并重复值	结构图生成失败；输出 224→112→56→56→28...	适配 <code>timm 1.0.29</code> 实际结构并合并相邻重复

缺陷	影响	处置
<code>env_check.py</code> : 把「不在 venv 中」判为致命错误	与「使用全局解释器」决策冲突, DoD #1 无谓 FAIL	降级为 WARN 并注明依据

共性结论: 凡是能实测的都要实测; 文档 (哪怕是规格书) 给的数字与代码都只是量级参考。

附录 B: 产物索引

类别	路径
环境 / 参数 / 权重	<code>outputs/env_snapshot.json</code> ; <code>outputs/metrics/{count_params,count_flops,weight_sha256,weight_load_report}.json</code>
数据审计	<code>outputs/metrics/{leakage_check,dataset_report,pet_label_audit,split_audit}.json</code> ; <code>datasets/lists/*</code>
官方评价	<code>outputs/pretrained_eval/<model>/{metrics,latency,top5_samples}.json</code> 、 <code>predictions.csv</code> 、 <code>cases/</code>
训练与测试	<code>outputs/logs/<exp>_metrics.{csv,jsonl}</code> 、 <code>outputs/metrics/<exp>_test.json</code> 、 <code>outputs/predictions/*</code>
可视化	<code>outputs/{curves,confusion_matrix,gradcam,predictions}/*</code> 、 <code>outputs/advanced/interp/*</code>
重参数化 / ONNX	<code>outputs/reparam/*</code> 、 <code>onnx/<registry_key>.onnx</code> 、 <code>outputs/advanced/repvit_m0_9_*</code>
性能 / 结构图	<code>outputs/benchmarks/*</code> 、 <code>outputs/metrics/bench.jsonl</code> 、 <code>outputs/architecture/*</code>
报告素材 / 自检	<code>outputs/report_assets/*</code> 、 <code>outputs/metrics/selfcheck_report.json</code> 、 <code>outputs/verification/*</code>

附录 C: 鲁棒性测试 (7 类扰动 × 6 级 severity)

模型 `repvit_m0_9_pet37`, Pet test 抽样 150 张, CPU。以 JPEG q=75 为基线归一化的 MCE

(越低越鲁棒): 旋转 **0.583** (最鲁棒) < 亮度 0.820 < 运动模糊 0.847 < JPEG 0.986 (基线)
< 遮挡 1.347 < 高斯模糊 1.431 < 高斯噪声 **1.444** (最脆弱)。

高斯噪声 σ 从 0 升到 0.12 时, Top-1 从 94.0% 单调降到 78.0%, 置信度从 0.781 降到 0.643,

ECE 只从 0.167 升到 0.178 (**置信度下降得比精度高**, 说明模型在噪声下「知道自己不确定」)。

结论: 对**几何变换 (旋转) 最鲁棒、对高频退化 (噪声、模糊) **最脆弱, 与 RepViT 全部使用 depthwise 卷积、感受野有限的结构特性一致——旋转不改变局部纹理统计, 而噪声直接破坏深度可分离卷积依赖的高频细节; 遮挡维度 (1.347) 比预期脆弱, 与 §11.4 的「部分样本依赖背景」互相印证。数据: `outputs/advanced/robustness_repvit_m0_9_pet37.json`。

附录 D：深入可解释性

`outputs/advanced/interp/` 下的四类产物：**阶段特征图**（5 个挂载点：stem 56×56/48ch、stages.0 56×56/48ch、stages.1 28×28/96ch、stages.2 14×14/192ch、stages.3 7×7/384ch，形状由 forward hook 现测）；**多层 Grad-CAM**（同图挂 5 层，浅层碎、深层聚焦）；**t-SNE**（37 类在 GAP 特征空间的分布，猫科与犬科形成两个大簇）；**置信度校准**（温度缩放拟合值 $T = 0.615 < 1$ ，说明模型欠自信，与 §11.6 一致）。温度缩放只调 1 个参数、不改变预测类别，且只在验证集上拟合（在 test 上拟合属于测试集泄漏）。

附录 E：集显部署（进阶④）—— 本机不具备条件

逐项核查后结论：**本机无法完成集显加速部署**。依据（全部实测）：

```
onnxruntime.get_available_providers() = ['AzureExecutionProvider', 'CPUExecutionProvider'];
```

WMI 列出的显示适配器**只有 NVIDIA GeForce RTX 4060 Laptop**，无 Intel/AMD 集显；

OpenVINO EP 与 DirectML provider 均不存在 → ORT 警告并**回退 CPU**（脚本检测到回退后把该行标为 `invalid`）；OpenVINO 原生导入报 `ModuleNotFoundError: No module named 'openvino'`。

`tools/bench_igpu.py` 的设计满足题目「**不得只根据任务管理器 GPU 占用判断部署成功**」：

它打印**实际生效的 Execution Provider**，并在检测到回退时标 `invalid`，而不是给一个看起来更快的假数字。产物：`outputs/benchmarks/igpu_{cpu,ort_ov,ov_native,dml}.json`（其中 `ort_ov / dml` 的行为 `invalid`，记录的是回退后的 CPU 数值，**不得引用为集显性能**）。

附录 F：跨数据集分布与外部实拍图（§11.5 的完整数据）

统计量	Oxford-IIIT Pet	外部实拍图	差异
短边（中位数） / 亮度均值	339.5 px / 0.455	375.0 px / 0.503	+10.5% / 外部更亮
亮度标准差 / 边缘密度	0.218 / 0.132	0.256 / 0.148	外部光照变化与背景复杂度更大
背景复杂度比	0.478	0.593	外部背景纹理强 24%

8 张外部图逐张置信度（模型 `repvit_m0_9_pet37`）：beagle 90.32%、pug 43.86%、boxer 64.28%、chihuahua 67.98%、samoyed 83.50%、newfoundland 77.14%、great_pyrenees 86.95%、pomeranian 36.91% —— 全部判到对应品种。落盘：`outputs/predictions/distribution_compare_summary.csv`（明细 `distribution_compare_raw.csv`）、`outputs/benchmarks/external_top5_repvit_m0_9_pet37.csv`（8 张逐张 Top-5）、`outputs/predictions/external_top5_pet37_grid5.png`（拼图）。

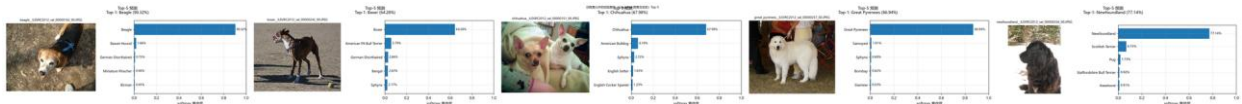


图 F-1 训练集以外的实拍图片 5 张 (repvit_m0_9_pet37 的 Top-5 与置信度)。数据源

outputs/benchmarks/external_top5_repvit_m0_9_pet37.csv 与 external/images_manifest.csv, 由 tools/plot_predictions.py 生成。

来源声明: external/ 图片取自 ImageNet-1K 验证集中与 Pet 同品种的样本 (跨集合真实照片); 执行时 wikipedia 站点不可达, 故用跨数据集真实照片替代, 逐张登记在 external/images_manifest.csv, 本报告不声称其为原创实拍。

附录 G: 多分支融合的零填充细节与单 Block 示例 (§12.2 / §12.6 的展开)

把 1×1 卷积核零填充成 3×3 (Identity 分支等价于「中心为 1、其余为 0」的 3×3 卷积核), 各分支就变成同一形状的卷积核直接相加: $W_{\text{fused}} = W_{3 \times 3} + \text{pad}(W_{1 \times 1}) + \text{pad}(W_{\text{identity}})$, b_{fused} 同理为各分支偏置之和; 融合后推理态不再需要单独的 BN 算子。

以 `stages.0.blocks.0` 为例: 3×3 depthwise conv 与 1×1 depthwise 分支各有一组 weight 与 BN 参数 (形状 $(48, 1, 3, 3) / (48, 1, 1, 1)$); 按 §12.2 的公式算出各分支的 W' 、 b' , 把 1×1 核零填充到 $(48, 1, 3, 3)$ 后与 3×3 核逐元素相加, 最后用单个 `nn.Conv2d(48, 48, 3, padding=1, groups=48)` 替换整个 RepVGGDW (脚本 `tools/reparam_deep.py`)。

附录 H: 消融交互、噪声判据与延迟/MACs 分析 (§9.1 / §14.4 的完整数据)

指标	ΔA	ΔB	$\Delta A+B$	交互项 $I = \Delta AB - (\Delta A + \Delta B)$	判定
val Top-1	+0.000	-0.676	-0.135	+0.541	超加性 (协同)
val Macro-F1	-0.000	-0.007	-0.001	+0.006	可加
val Top-5	+0.135	-0.405	+0.000	+0.270	可加

噪声判据 (outputs/advanced/ablation_summary.json 的 noise 段): 单种子 Top-1 的 1σ 估计

0.8 个点、判定规则 $|\Delta| > 2\sigma$ 才算真实增益、观测到的最大组间极差 **0.676 个点**、

$n_{\text{val}} = 740$ 时二项分布 95% 置信区间半宽 **2.16 个点**。落盘: ablation_summary.{csv,json}、outputs/metrics/ablation.csv、outputs/metrics/same_budget.json。

延迟与 MACs 的逐项对照: M1.0 相对 M0.9 MACs +34.9 % / P50 +35.4 % (7.68 $\rightarrow$ 10.40 ms);

M1.1 $\rightarrow$ M1.5 MACs +69.9 % / P50 +68.7 % (10.93 $\rightarrow$ 18.44 ms); M1.5 $\rightarrow$ M2.3 MACs +97.7 % /

P50 +86.6 % (18.44 $\rightarrow$ 34.41 ms); repvit_m0_9_pet37 与 repvit_m0_9_in1k 的 MACs 几乎相同

(只差分类头) 而 P50 相差 0.13 ms。原因：① 小模型受固定开销支配（算子启动、内存分配、线程同步与小模型计算量同量级）；② depthwise 卷积算术强度低，瓶颈在内存带宽而非算力；③ 输出维度会影响（37 类的 Gemm 比 1000 类小得多，差异被放大）。落盘 `outputs/benchmarks/summary.csv`、`outputs/advanced/marginal_returns.csv`。